

AULA 3/4

- Exemplos de eventos:
 - Página HTML terminou o carregamento
 - Um elemento HTML foi alterado
 - Um botão HTML foi clicado
 - Etc ...
- Com o JS é possível interceptar estes eventos e aplicar alguma ação quando eles acontecerem.

```
<element event='some JavaScript'></element>
```

JS: eventos

```
<!DOCTYPE html>
<html>
<body>
  <button onClick="document.getElementById( 'demo' ).innerHTML=Date()">The
time is?</button>

  <p id="demo"></p>
</body>
</html>
```

```
<!DOCTYPE html>
<html>
<body>
  <button onClick="this.innerHTML=Date()">The time is?</button>
</body>
</html>
```

- Forma mais elegante...

```
<!DOCTYPE html>
<html>
<body>
  <p>Click the button to display the date.</p>
  <button onClick="displayDate()">The time is?</button>
  <script>
    function displayDate() {
      document.getElementById("demo").innerHTML = Date();
    }
  </script>
  <p id="demo"></p>
</body>
</html>
```

JS: eventos comuns

Evento	Descrição
onchange	Um elemento HTML mudou
onclick	O elemento HTML foi clicado
onmouseover	O mouse passou por cima do elemento HTML
onmouseout	O mouse saiu de cima do elemento HTML
onkeydown	O usuário apertou uma tecla
onload	O navegador terminou de carregar a página

A SMALL TASTE

- clicks
 - drags
 - drops
 - hovers
 - scrolls
 - form submission
 - key presses
 - focus/blur
- 
- A cartoon illustration of a person with dark hair, wearing an orange sweater and a white and black checkered apron, cooking at a red stove. They are holding a black spoon and stirring a large black pot. Steam is rising from the pot.
- mouse wheel
 - double click
 - copying
 - pasting
 - audio start
 - screen resize
 - printing

JS: eventos comuns

```
<body onload="myFunction()">
```

```

```

```
<script>
function loadImage() {
    alert("Image is loaded");
}
</script>
```

```
<input type='text' id='num2' onchange="recalcular()">
```

JS: addEventListener

```
<button id="meu-botao">Clique em mim</button>
```

```
<script>
```

```
// Passo 1: Pegamos o botão no HTML e guardamos em uma variável
```

```
let botao = document.getElementById("meu-botao");
```

```
// Passo 2: Adicionamos o ouvinte de eventos (o "fofoqueiro")
```

```
botao.addEventListener("click", () => {
```

```
  // Passo 3: O que acontece quando o clique ocorrer?
```

```
  botao.innerText = "Você clicou no botão!";
```

```
  botao.style.backgroundColor = "green";
```

```
  botao.style.color = "white";
```

```
});
```

```
</script>
```


JS: addEventListener

```
// --- FUNÇÃO 2: Focada em emitir avisos ---  
botao.addEventListener("click", () => {  
    console.log("O botão foi clicado! Registrando no sistema...");  
    alert("Esta é a segunda função rodando no mesmo clique!");  
});
```

O `addEventListener` permite associar mais de uma função ao mesmo evento. Por exemplo, `fun1` e `fun2`, usando somente o `onclick = fun1`, nunca irá chamar as duas funções, só se colocássemos elas aninhadas.

Com o `addEventListener`, cada chamada ao método é um novo evento/função adicionado.

Como terceiro parâmetro, também é possível passar “options”. Por exemplo, `{once: true}` vai permitir que a função execute somente 1x, após isso, é removida da lista de funções a serem executadas no evento.

- Evite usar variáveis globais
 - Minimize o uso de variáveis globais.
 - Evite também objetos e funções globais.
 - Variáveis globais podem ser sobrescritas por outros trechos de código, levando ao caos.
- Use variáveis locais
 - Variáveis locais devem ser declaradas com `var` ou `let`, caso contrário, serão variáveis globais.
 - `var` tem escopo de função
 - `let` tem escopo de bloco

JS: BOAS PRÁTICAS

```
<script>
  var x = 10;
  // Here x is 10

  {
    var x = 2;
    // Here x is 2
  }
  // Here x is 2
</script>
```

```
<script>
  var x = 10;
  // Here x is 10

  {
    let x = 2;
    // Here x is 2
  }
  // Here x is 10
</script>
```

- Coloque todas as declarações no começo de cada trecho de código
 - Código limpo e claro
 - Local único para saber quais variáveis há
 - Fácil identificar o uso de variáveis globais
 - Reduz o risco de reescrever uma variável

JS: BOAS PRÁTICAS

```
// Declare no começo
var firstName, lastName, price, discount, fullPrice;

// Use depois
firstName = "John";
lastName = "Doe";

price = 19.90;
discount = 0.10;

fullPrice = price * 100 / discount;

// TAMBÉM VALE PARA LOOPS. Declare no começo.
var i;

// Use depois
for (i = 0; i < 5; i++) {}
```

- Inicialize as variáveis após a declaração
 - Provê código limpo
 - Evita valores undefined
 - Único lugar para consultar as inicializações

```
// Declare e inicialize no início
var firstName = "",
    lastName = "",
    price = 0,
    discount = 0,
    fullPrice = 0,
    myArray = [],
    myObject = {};
```

- Não declare tipos básicos com tipos complexos
 - Exemplo: não use `new String("")`, isso irá criar um `Object`
- Use `{}` ao invés de `new Object()`
- Use `""` ao invés de `new String()`
- Use `0` ao invés de `new Number()`
- Use `false` ao invés de `new Boolean()`
- Use `[]` ao invés de `new Array()`
- Use `function (){}` ao invés de `new Function()`

```
var x1 = {};  
var x2 = "";  
var x3 = 0;  
var x4 = false;  
var x5 = [];  
var x6 = /()/;  
var x7 = function () { }; // new function object
```

- Cuidado com as conversões de String e Number

```
var x = 5 + 7;           // x.valueOf() is 12,    typeof x is a number
var x = 5 + "7";        // x.valueOf() is 57,    typeof x is a string
var x = "5" + 7;        // x.valueOf() is 57,    typeof x is a string
var x = 5 - 7;          // x.valueOf() is -2,    typeof x is a number
var x = 5 - "7";        // x.valueOf() is -2,    typeof x is a number
var x = "5" - 7;        // x.valueOf() is -2,    typeof x is a number
var x = 5 - "x";        // x.valueOf() is NaN,   typeof x is a number
```


- Use `===` ao invés de `==`
 - O operador `==` converte automaticamente os valores para o mesmo tipo antes de verificar
 - O operador `===` força que os tipos sejam iguais

```
0 == "";           // true
1 == "1";          // true
1 == true;          // true

0 === "";           // false
1 === "1";          // false
1 === true;         // false
```

- Use valores default para suas variáveis, um valor undefined pode quebrar o seu programa

```
function myFunction(x, y) {  
  if (y === undefined) {  
    y = 0;  
  }  
}  
  
// ou  
  
function (a = 1, b = 1) { /*function code*/ }
```

- Sempre termine os Switches com o valor default

```
switch (new Date().getDay()) {  
  case 0:  
    day = "Sunday";  
    break;  
  case 1:  
    day = "Monday";  
    break;  
  case 2:  
    day = "Tuesday";  
    break;  
  case 3:  
    day = "Wednesday";  
    break;  
  case 4:  
    day = "Thursday";  
    break;  
  case 5:  
    day = "Friday";  
    break;  
  case 6:  
    day = "Saturday";  
    break;  
  default:  
    day = "Unknown";  
}
```

- Evite usar a função eval()
 - Esta função serve para executar código texto como código JS, pode levar a diversos problemas de segurança...
- Reduza a quantidade de comandos dentro dos loops

```
var i;  
for (i = 0; i < arr.length; i++) { }  
  
//Melhor usar assim:  
var i;  
var l = arr.length;  
for (i = 0; i < l; i++) {?
```

JS: eval()

```
let textoDigitado = "10 * 5 + 2";  
// O eval lê o texto, percebe que é uma conta matemática  
e executa!  
let resultado = eval(textoDigitado);  
console.log(resultado); // Vai imprimir: 52
```

Exemplo:
Calculadora
com eval()

```
let textoDigitado = "document.body.innerHTML = '<h1>VOCÊ  
FOI HACKEADO</h1>'";  
// O seu sistema confia na entrada e roda o eval:  
eval(textoDigitado);
```

JS: eval()

```
let textoDigitado = "10 + 5";  
// Cortamos o texto onde tem o "+"  
let partes = textoDigitado.split("+");  
// Agora 'partes' é uma lista (array) com dois textos: ["10", " 5"]  
// Transformamos os textos em números reais e somamos de verdade  
let numero1 = parseFloat(partes[0]); // 10  
let numero2 = parseFloat(partes[1]); // 5  
let resultado = numero1 + numero2;  
console.log(resultado); // Imprime: 15
```